

passgen

Детерминированный генератор паролей

Документация для защиты проекта

Подробное описание всех функций человеческим языком

1. Что такое passgen и зачем он нужен?

passgen — это программа, которая создает надёжные пароли на основе одной секретной фразы. Вам не нужно запоминать 20 разных паролей для разных сайтов. Достаточно запомнить одну фразу (сид), а программа сама вычислит пароль для каждого сервиса.

Проблема, которую решает программа: в современном мире у каждого человека десятки аккаунтов — почта, соцсети, банки, работа, учёба. Использовать один пароль везде опасно. Запоминать 30 разных сложных паролей невозможно. Менеджеры паролей хранят все пароли в одном месте, но если их взломают — украдут всё.

Решение от passgen: пароль не хранится, а вычисляется каждый раз заново по алгоритму HMAC-SHA256. Если вы знаете сид ("МояСекретнаяФраза") и название сервиса ("google"), то можете восстановить пароль в любой момент на любом компьютере. Пароль невозможно потерять или забыть — он всегда будет таким же для тех же входных данных.

2. Главный алгоритм: как программа создаёт пароль?

Самый важный вопрос на защите: как именно программа создает пароль? Объяснение по шагам:

Шаг 1. Сид — это секретный ключ

Пользователь вводит любую фразу, например "МояСуперФраза2024". Программа превращает эту фразу в набор чисел (байтов) и использует как ключ для криптографического алгоритма HMAC. Без этого ключа невозможно вычислить пароль.

Шаг 2. Формируется сообщение

Программа берёт название сервиса ("google"), добавляет к нему нужную длину пароля (16) и номер шага (1, 2, 3...). Получается одно сообщение. Номер шага нужен, чтобы пароль мог быть длиннее 32 символов.

Шаг 3. Вычисляется HMAC-SHA256

Это международный криптографический стандарт. Он используется в банковских системах, в блокчейне биткоина, в защите Wi-Fi. Алгоритм берёт ключ (сид) и сообщение и выдаёт 256 бит случайных данных. Важное свойство: для одинаковых ключа и сообщения результат всегда одинаковый. Это называется детерминированность.

Шаг 4. Байты превращаются в символы

Из 32 полученных байтов программа берёт каждый, проверяет, подходит ли он (чтобы не было перекоса вероятности), и превращает в символ из выбранного набора. Например, байт 65 может стать буквой "B", а байт 7 — цифрой "7". Процесс повторяется, пока не наберётся нужная длина пароля.

Шаг 5. Применяются дополнения

Если включена "Первая заглавная" — первый символ становится заглавным. Если включена "Последняя цифра" — последний символ заменяется на цифру (вычисляется отдельно, чтобы сохранить детерминированность). Это нужно, потому что многие сайты требуют заглавную букву и цифру в пароле.

Итог: даже если злоумышленник узнал пароль для "google", он не сможет вычислить пароль для "github" — названия разные, результаты HMAC будут совершенно разными. А если вы забыли пароль — просто введите тот же сид и название сервиса, программа выдаст точно такой же пароль.

3. Почему программа безопасна?

Пять причин доверять программе:

1. Все данные остаются на вашем компьютере

Сервер работает на localhost (127.0.0.1). Весь интерфейс открывается с адреса вашего компьютера. Никакие данные не уходят в интернет. Никто не может перехватить ваш сид или пароль.

2. Используется стандартная криптография, а не самодельная

HMAC-SHA256 — это международный стандарт (RFC 2104), а не выдумка авторов программы. Он проверен миллионами специалистов по всему миру, используется в банках, армии, криптовалютах. Программа не изобретает велосипед — она использует готовый надёжный инструмент.

3. Нечего красть — пароль не хранится

Пароль не записывается в базу данных. Он вычисляется каждый раз заново. Если хакер получит доступ к компьютеру, он найдёт только файлы программы, но не пароли. А сессионный режим гарантирует, что даже настройки удаляются при закрытии окна.

4. Равномерное распределение символов

Программа использует технику rejection sampling: неподходящие байты отбрасываются. Это гарантирует, что все символы встречаются с одинаковой вероятностью. Без этого одни символы появлялись бы чаще других, и пароль было бы легче угадать.

5. Для каждого сервиса — свой пароль

Название сервиса участвует в вычислении. Измените одну букву — получите совершенно другой пароль. Это защищает от ситуации: узнали пароль от Google — не смогут войти в GitHub.

4. Все элементы интерфейса: для чего каждый?

В этом разделе — объяснение каждого элемента программы. Зачем он нужен, какую проблему решает, как им пользоваться.

Поле "Сид"

Самое важное поле. Сюда вводится секретная фраза — единственное, что нужно запомнить. Без сида программа не сможет восстановить пароль. Если поле пустое, появляется розовая подсветка и сообщение об ошибке. Это сделано специально: лучше сразу сказать пользователю об ошибке, чем дать сгенерировать бессмысленный пароль.

Поле "Сервис"

Сюда вводится название сайта или сервиса: google, github, vk, yandex и т.д. Алгоритму HMAC нужно различать пароли — поэтому используется название. Справа есть кнопка "список" с популярными сервисами (Google, GitHub, VK, Яндекс и др.). У каждого сервиса цветной кружок-логотип для быстрого визуального поиска. Если сервиса нет в списке — можно напечатать своё название. Программа запомнит его.

Длина пароля

Сколько символов будет в пароле. Чем длиннее — тем сложнее взломать. Но слишком длинный пароль неудобно вводить. Рекомендуемая длина: 10-16 символов. Стрелочки рядом с полем позволяют быстро менять значение без клавиатуры.

Набор символов — четыре кружочка

Заглавные буквы, строчные, цифры, спецсимволы. Многие сайты требуют определённые типы символов. Программа даёт гибкость: включите только то, что нужно. Спецсимволы (!@#\$%) выключены по умолчанию, потому что некоторые сайты их не принимают.

Пресеты — быстрые настройки

Три готовых варианта для тех, кто не хочет разбираться: Минимальный (8 символов, буквы+цифры), Стандартный (10 символов, всё кроме спецсимволов), Максимальный (16 символов, всё включено). Выбрал — и готово.

Расширенные настройки

"Первая заглавная" и "Последняя цифра" спрятаны, чтобы не занимать место, так как используются реже. Многие сайты требуют заглавную букву и цифру — эти опции гарантируют их наличие.

Поле логина

Многие сервисы просят придумать логин. Программа генерирует короткий (6 символов) логин, который легко запомнить. Используются только "непутающиеся" буквы — нет i, l, o (их часто путают с 1 и 0). Каждое нажатие кнопки даёт новый логин.

Полоска надёжности

Три полоски под паролем показывают, насколько он надёжен. Розовый (слабый) -> малиновый (средний) -> сиреневый (отличный). Это мгновенная обратная связь: пользователь видит, нужно ли увеличить длину или добавить типы символов.

Кнопка "Создать"

Главная кнопка. Запускает алгоритм HMAC-SHA256. После нажатия пароль автоматически копируется в буфер обмена. Enter в полях сида или сервиса тоже срабатывает как нажатие "Создать".

Кнопка "Копировать"

На случай, если нужно скопировать пароль повторно. Использует стандартный API буфера обмена браузера.

Экспорт / Импорт

Экспорт скачивает настройки в JSON-файл. Импорт загружает их обратно с выбором — какие сервисы импортировать. Удобно при переходе на другой компьютер.

5. Описание каждой функции программы

Этот раздел — ключевой для защиты. Каждая функция объясняется по схеме: ЗАЧЕМ нужна (какую проблему решает), ГДЕ используется, КАК работает.

init() — подготовка программы к работе

ЗАЧЕМ: Когда пользователь открывает страницу, нужно настроить все поля, кнопки и списки. Это как включение компьютера — все системы должны быть готовы.

ГДЕ: Запускается автоматически при загрузке.

КАК: Очищает старые настройки (безопасность), привязывает обработчики событий (Enter для генерации, снятие ошибки при вводе сида), восстанавливает настройки текущего сервиса или сбрасывает на стандартные.

loadState() — очистка данных для безопасности

ЗАЧЕМ: Чтобы каждый запуск начинался с чистого листа. Если компьютером пользуется кто-то другой, он не увидит, какие сервисы вы использовали.

ГДЕ: Вызывается из init() при каждой загрузке.

КАК: Полностью очищает все данные: список сервисов, настройки, длину. Удаляет даже старые сохранения из localStorage, если они остались от предыдущих версий.

saveCfg() — запоминание настроек сервиса

ЗАЧЕМ: Чтобы при повторном выборе сервиса не пришлось заново всё настраивать. Программа запоминает, какие настройки вы использовали.

ГДЕ: Вызывается при нажатии Создать и при выборе пресета.

КАК: Берёт текущие значения чекбоксов, длины, инверсии и сохраняет их в памяти под названием сервиса. Важно: не сохраняется на диск — только в оперативной памяти.

applyCfg(cfg) — восстановление настроек

ЗАЧЕМ: Когда вы кликаете на сервис в списке, программа должна вспомнить его настройки.

ГДЕ: При клике на сервис, при автоматическом восстановлении.

КАК: Достает сохранённые настройки, устанавливает длину, включает/выключает нужные чекбоксы и опции инверсии. Всё как было.

toggleInv(id) — переключение первой заглавной/последней цифры

ЗАЧЕМ: Многие сайты требуют заглавную букву и цифру. Включаем — и пароль автоматически их содержит.

ГДЕ: При клике на соответствующие пункты в расширенных настройках.

КАК: Меняет значение на противоположное (0->1, 1->0) и обновляет кружочек на экране.

applyPreset() — быстрые настройки одним кликом

ЗАЧЕМ: Чтобы пользователь мог быстро выбрать готовый вариант, не разбираясь.

ГДЕ: При выборе в выпадающем списке пресетов.

КАК: Устанавливает длину и включает символы по одному из трёх сценариев: минимальный (8), стандартный (10), максимальный (16).

renderChk() — отрисовка кружочков выбора символов

ЗАЧЕМ: Чтобы показать, какие типы символов выбраны, и дать возможность их менять.

ГДЕ: При загрузке, при смене сервиса, при клике.

КАК: Создаёт четыре строки с кружочками. Если набор активен — кружок фиолетовый. При клике значение переключается.

adj(delta) — изменение длины пароля

ЗАЧЕМ: Быстрая подстройка длины стрелочками, без клавиатуры.

ГДЕ: При клике на стрелки вверх/вниз.

КАК: Прибавляет или отнимает 1 от текущей длины. Следит, чтобы длина была от 1 до 99.

toggleSvcPanel() — показать/спрятать список сервисов

ЗАЧЕМ: Список занимает много места, поэтому он скрыт и показывается по клику.

ГДЕ: При клике на кнопку 'список'.

КАК: Если скрыт — показывает с анимацией. Если открыт — скрывает. Стрелка поворачивается.

renderSvcChips() — сборка списка сервисов

ЗАЧЕМ: Показать все доступные сервисы: популярные и те, что пользователь добавлял.

ГДЕ: При загрузке, при добавлении сервиса, при импорте.

КАК: Сначала популярные сервисы (которых нет в сохранённых), затем все сохранённые.

chip() — создание кнопки сервиса

ЗАЧЕМ: Каждый сервис — отдельная кнопка с названием и логотипом.

ГДЕ: Вызывается из renderSvcChips() для каждого сервиса.

КАК: Если сервис из популярного списка — цветной кружок с буквой (Google = синий G). Если пользовательский — серый кружок с ?. По клику заполняет поле сервиса. Правый клик на сохранённом сервисе удаляет его.

gen() — ГЛАВНАЯ ФУНКЦИЯ генерации пароля

ЗАЧЕМ: Самая важная функция. Ради неё написана вся программа. Создаёт пароль.

ГДЕ: При нажатии Создать или Enter.

КАК: Проверяет сид (не пустой). Проверяет чекбоксы (хотя бы один). Составляет алфавит. Вызывает hmacGen() для HMAC-SHA256. Применяет первую заглавную и последнюю цифру. Показывает результат. Обновляет полосу надёжности. Копирует в буфер обмена. Добавляет сервис в список. Сохраняет настройки.

hmacGen() — криптографическое ядро

ЗАЧЕМ: Здесь происходит actual криптография — вычисление HMAC-SHA256.

ГДЕ: Вызывается из gen().

КАК: Создаёт ключ HMAC из сида. Формирует сообщение (сервис+длина+счётчик). Вычисляет HMAC (32 байта). Отбрасывает неподходящие байты (rejection sampling). Превращает подходящие в символы. Повторяет, пока не наберётся нужная длина.

genLogin() — генерация логина

ЗАЧЕМ: Для сервисов, где нужен не только пароль, но и логин/имя пользователя.

ГДЕ: При нажатии кнопки Создать логин.

КАК: Вычисляет SHA-256 от сида + 'login' + сервис + счётчик. Берёт 6 байтов и превращает в символы. Алфавит: буквы без i,l,o (чтобы не путать с 1,0) и цифры 2-9. Счётчик растёт с каждым кликом — каждый раз новый логин.

updateStrength() — оценка надёжности

ЗАЧЕМ: Пользователь должен сразу видеть, хороший пароль или нужно улучшить.

ГДЕ: После генерации пароля.

КАК: Если длина < 10 или меньше 3 наборов — слабый (розовый). Если длина >= 10 и 3+ набора — средний (малиновый). Если длина >= 14 и 4 набора — отличный (сиреневый). Показывает текстовую рекомендацию.

сру() — копирование в буфер обмена

ЗАЧЕМ: Скопировать пароль, чтобы вставить на сайте.

ГДЕ: При нажатии Копировать или автоматически после генерации.

КАК: Пытается использовать современный Clipboard API. Если не сработало — старый метод execCommand.

Показывает 'Скопировано!' на 1.5 секунды.

exportData() — экспорт настроек

ЗАЧЕМ: Перенести настройки на другой компьютер или сделать резервную копию.

ГДЕ: При клике на Экспорт.

КАК: Собирает все сервисы с настройками, форматирует в JSON с отступами, создаёт файл и скачивает его.

importData() — импорт настроек

ЗАЧЕМ: Загрузить ранее сохранённые настройки.

ГДЕ: При выборе JSON-файла через Импорт.

КАК: Читает файл, проверяет JSON, показывает окно выбора сервисов. После подтверждения загружает настройки.

showImportModal() — окно выбора сервисов для импорта

ЗАЧЕМ: Чтобы пользователь мог выбрать, какие сервисы импортировать.

ГДЕ: Из importData() после чтения файла.

КАК: Создаёт список с чекбоксами для каждого сервиса из файла. Все включены по умолчанию. Показывает окно поверх программы.

confirmImport() — подтверждение импорта

ЗАЧЕМ: Записать выбранные настройки в программу.

ГДЕ: При нажатии Импортировать в модальном окне.

КАК: Берёт отмеченные сервисы, загружает настройки, добавляет в список. Показывает сообщение об успехе.

closeImportModal() — отмена импорта

ЗАЧЕМ: Если пользователь передумал.

ГДЕ: При нажатии Отмена.

КАК: Просто закрывает окно и очищает временные данные.

toggleAdvanced() — расширенные настройки

ЗАЧЕМ: Опции инверсии используются реже и спрятаны, чтобы не занимать место.

ГДЕ: При клике на Расширенные.

КАК: Раскрывает или сворачивает панель с анимацией. Стрелка поворачивается.

6. Экспорт и импорт

Программа умеет сохранять настройки в файл и загружать их обратно. Формат файла — JSON (текстовый, читается человеком).

Зачем это нужно?

Представьте: вы настроили пароли для 20 сервисов на домашнем компьютере. Теперь нужно настроить то же самое на ноутбуке. Вместо того чтобы заново выставлять все настройки, экспортируйте их в файл, перенесите на ноутбук и импортируйте. Все настройки восстановятся.

Как выглядит файл?

```
{
  "svcs": ["google", "github"],
  "perSvc": {
    "google": {
      "state": {"capitals":1, "lowercase":1, "digits":0, "symbols":1},
      "inv": {"cap":0, "dig":1},
      "len": 16
    }
  }
}
```

svcs — список всех сервисов.

perSvc — настройки для каждого: какие символы (1/0), опции инверсии, длина.

Как проходит импорт?

При выборе файла программа показывает окно со списком всех сервисов из этого файла. Вы отмечаете только те, что нужны. Например, в файле 30 сервисов, а вам нужно 3 — отмечаете их и нажимаете Импортировать. Остальные игнорируются.

7. Установка и запуск

Программа работает на любой ОС с Python 3 (любая версия от 3.6 и выше). Не требует установки дополнительных пакетов.

Linux (Arch, Ubuntu и другие):

1. Скачать программу:

```
git clone ...  
cd xxxulay
```

2. Запустить:

```
python3 app.py
```

Windows:

1. Скачать ZIP с GitHub (Code -> Download ZIP), распаковать
2. Открыть папку и запустить:

```
python app.py
```

Что произойдёт?

1. В терминале появится: passgen — http://localhost:...
2. Автоматически откроется окно программы (отдельное окно, без адресной строки)
3. Если окно не открылось — перейдите по адресу из терминала в любом браузере
4. Для выхода: Ctrl+C в терминале

Зависимости

Не требуются. Программа использует:

- Стандартную библиотеку Python (http.server, socketserver) для сервера
- Web Crypto API браузера для криптографии
- HTML/CSS/JavaScript для интерфейса